

Quadcopter Dynamic Modeling and Control

Traditional Applications of UAV

- Military
- Border



Credit: ~~dronesfun, unmanned.co.uk, marketinfogroup~~

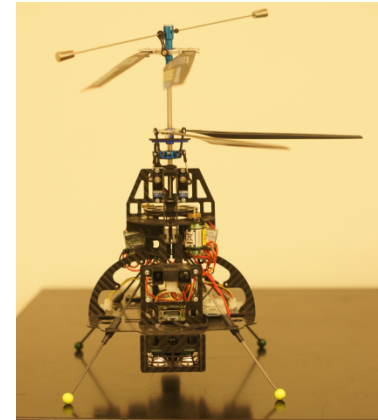
Civilian Drones in the Media



Kinds of UAVs



Single-rotor



Co-axial dual rotor



Fixed wing UAVs



Quadcopter



Octo-copter

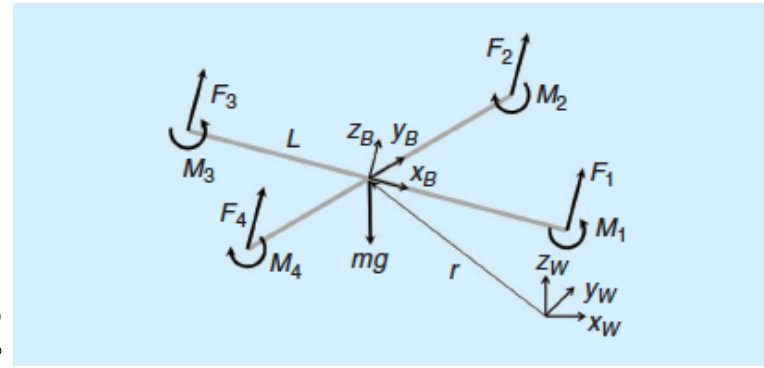
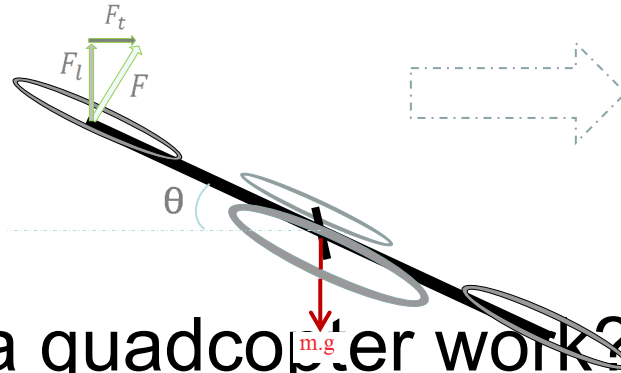
Rotating wing UAVs:
Rotorcrafts

Credit: Vision, Logos

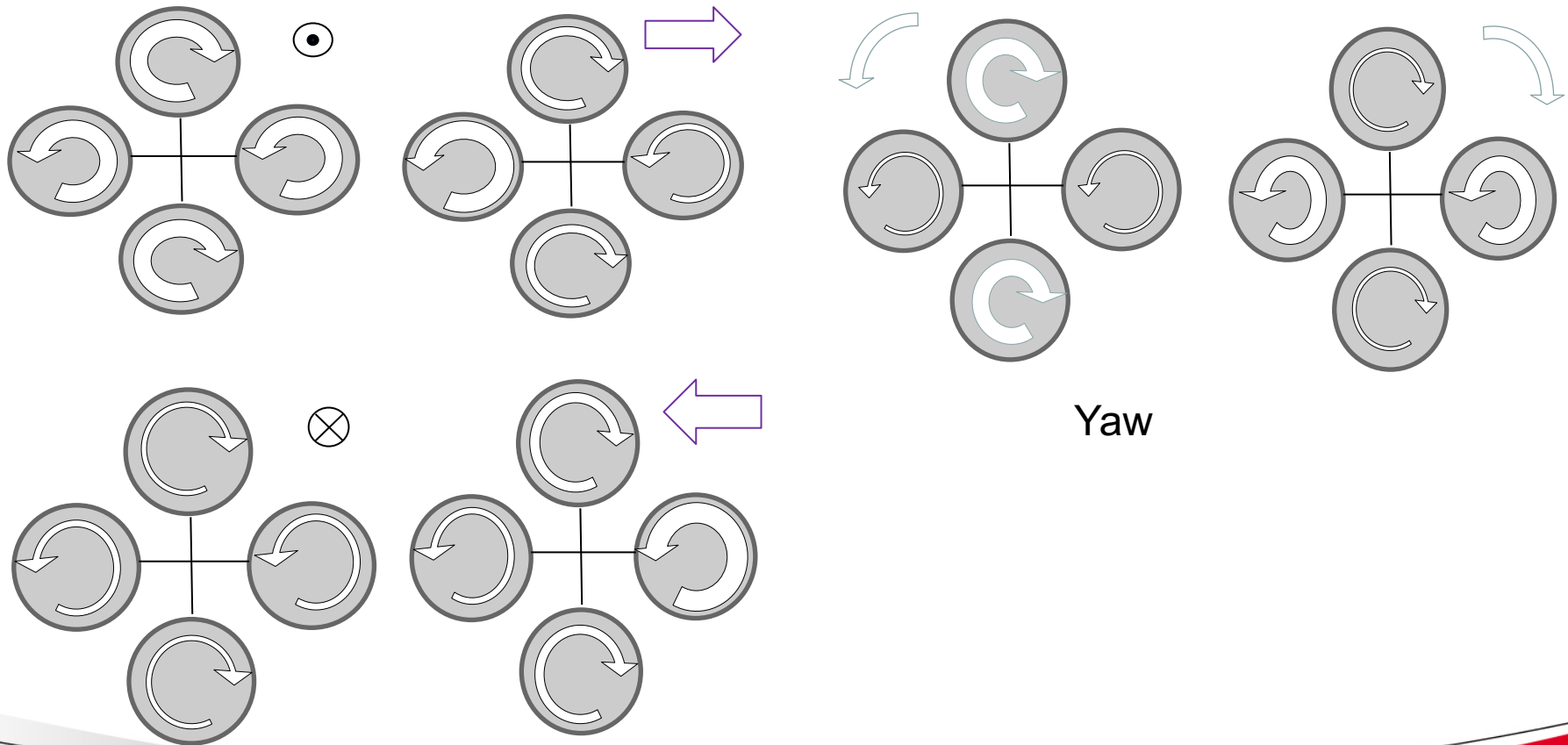
Introduction

- What is a Quad-Copter?
- An aerial vehicle using 4 rotors, Quad-copters are symmetrical vehicles with four equally sized rotors at the end of four equal length rods and typically using only the varying speed of the motors to achieve both stability and flight.
- Simplicity of construction and relying on fixed pitch rotors which cause the simplicity in maintenance.
- Maneuverability
- Ability to negotiate tight spaces





How does a quadcopter work?



Up & Down

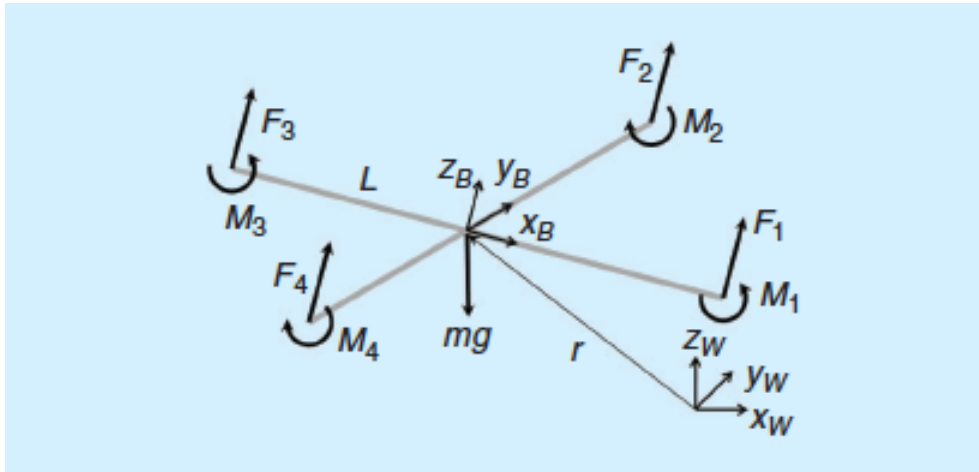
Left & Right

Yaw

Challenges in UAV Navigation

- Navigation consists of two parts:
 - Path planning: obtaining path that would take the UAV from point A to point B
 - Path tracking: obtaining controls to allow UAV to adhere to the planned path
- Carried out using feedback control
 - Requires knowledge of current location to steer the UAV
 - Feedback obtained with the help of sensors such as GPS, IMU, sonar, and laser

Dynamic Modeling



We define two frames of references:

i. The body frame - B

x_B is aligned towards the preferred forward direction, z_B is aligned towards the vertically up perpendicular to the plane of motion

ii. The inertial or world reference frame - W

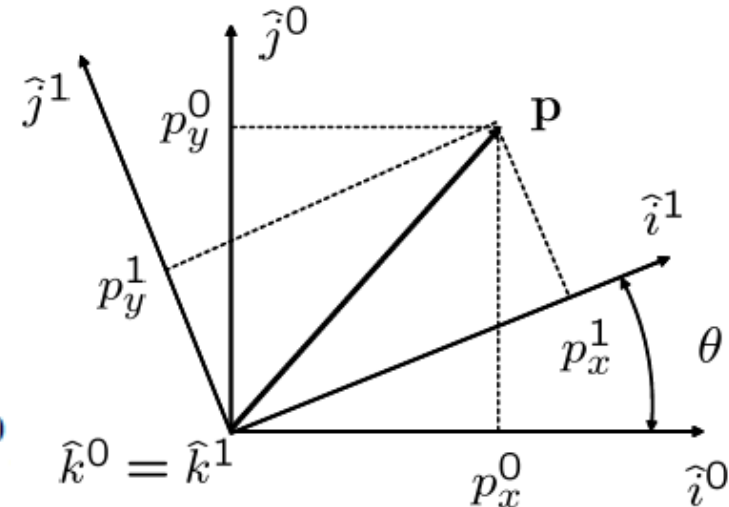
Rotation Matrices

- Consider a vector $\mathbf{p}$ in two coordinate frames '0' and '1'

In frame '0' $\mathbf{p} = p_x^0 \hat{i}^0 + p_y^0 \hat{j}^0 + p_z^0 \hat{k}^0$

In frame '1' $\mathbf{p} = p_x^1 \hat{i}^1 + p_y^1 \hat{j}^1 + p_z^1 \hat{k}^1$

$$p_x^1 \hat{i}^1 + p_y^1 \hat{j}^1 + p_z^1 \hat{k}^1 = p_x^0 \hat{i}^0 + p_y^0 \hat{j}^0 + p_z^0 \hat{k}^0$$



$$\mathbf{p}^1 \triangleq \begin{pmatrix} p_x^1 \\ p_y^1 \\ p_z^1 \end{pmatrix} = \begin{pmatrix} \hat{i}^1 \cdot \hat{i}^0 & \hat{i}^1 \cdot \hat{j}^0 & \hat{i}^1 \cdot \hat{k}^0 \\ \hat{j}^1 \cdot \hat{i}^0 & \hat{j}^1 \cdot \hat{j}^0 & \hat{j}^1 \cdot \hat{k}^0 \\ \hat{k}^1 \cdot \hat{i}^0 & \hat{k}^1 \cdot \hat{j}^0 & \hat{k}^1 \cdot \hat{k}^0 \end{pmatrix} \begin{pmatrix} p_x^0 \\ p_y^0 \\ p_z^0 \end{pmatrix}$$

$$\mathbf{p}^1 = \mathbf{R}_0^1 \mathbf{p}^0$$

Rotation Matrices

- Rotation along 'z'

$$R_{z,\theta} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- Rotation along 'x'

$$R_{x,\theta} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta \\ 0 & \sin \theta & \cos \theta \end{bmatrix}$$

- Rotation along 'y'

$$R_{y,\theta} = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

Quadcopter rotation matrix from body to world frame

- We use Z-X-Y Euler angles to model the rotation of quadcopter
 - To get from W to B, we first rotate about z_W by the yaw angle, ψ , then rotate about the intermediate x axis by roll angle, ϕ , and finally rotate by y_B axis by pitch angle, θ .

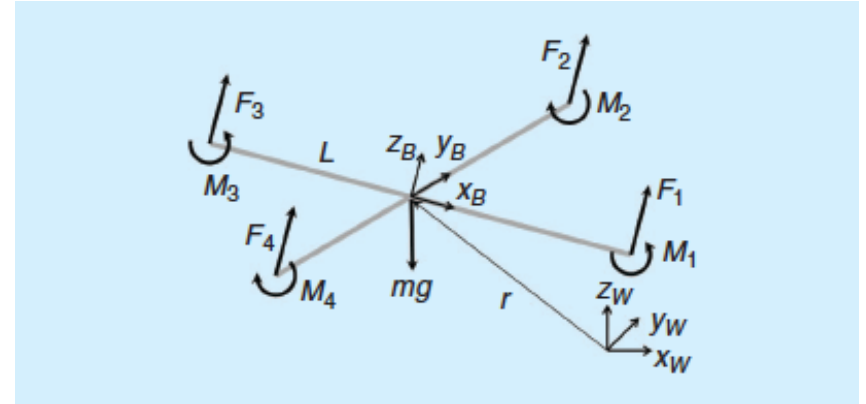
$$\begin{aligned}
 R &= \begin{bmatrix} c\psi & -s\psi & 0 \\ s\psi & c\psi & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & c\phi & -s\phi \\ 0 & s\phi & c\phi \end{bmatrix} \begin{bmatrix} c\theta & 0 & s\theta \\ 0 & 1 & 0 \\ -s\theta & 0 & c\theta \end{bmatrix} \\
 &= \begin{bmatrix} c\psi c\theta - s\phi s\psi s\theta & -c\phi s\psi & c\psi s\theta + c\theta s\phi s\psi \\ c\theta s\psi + c\psi s\phi s\theta & c\phi c\psi & s\psi s\theta - c\psi c\theta s\phi \\ -c\phi s\theta & s\phi & c\phi c\theta \end{bmatrix}
 \end{aligned}$$

Dynamic Modeling

Equations of Motion:

Forces:

$$m\ddot{\mathbf{r}} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + R \begin{bmatrix} 0 \\ 0 \\ \Sigma F_i \end{bmatrix}$$



Moments:

$$I \begin{bmatrix} \dot{p} \\ \dot{q} \\ \dot{r} \end{bmatrix} = \begin{bmatrix} L(F_2 - F_4) \\ L(F_3 - F_1) \\ M_1 - M_2 + M_3 - M_4 \end{bmatrix} - \begin{bmatrix} p \\ q \\ r \end{bmatrix} \times I \begin{bmatrix} p \\ q \\ r \end{bmatrix}$$

Where p, q, r are the angular velocities related to the roll, pitch, and yaw rates as:

$$\begin{bmatrix} p \\ q \\ r \end{bmatrix} = \begin{bmatrix} c\theta & 0 & -c\phi s\theta \\ 0 & 1 & s\phi \\ s\theta & 0 & c\phi c\theta \end{bmatrix} \begin{bmatrix} \dot{\phi} \\ \dot{\theta} \\ \dot{\psi} \end{bmatrix}$$

Motor Model

- Each rotor provides a force and a momentum according to:

$$F_i = k_F \omega_i^2, \quad M_i = k_M \omega_i^2$$

- Motor follows the following first order model

$$\dot{\omega}_i = k_m (w_i^{\text{des}} - w_i).$$

Quadcopter Control

- Before we control quadcopter, let us look at the hovering state:

$$\theta = 0, \phi = 0, \psi = \psi_0, r = r_0, \dot{r} = 0, \dot{\theta} = \dot{\phi} = \dot{\psi} = 0$$

- At hovering, the total vertical forces should equal to the weight:

$$\sum F_i = mg$$

Nominal hovering speed: $\omega_{i,0} = \omega_h = \sqrt{\frac{mg}{4k_F}}$

Quadcopter Control – Position Control

- Trajectory tracking: PID Controller

$$(\ddot{r}_{i,T} - \ddot{r}_i^{\text{des}}) + k_{d,i}(\dot{r}_{i,T} - \dot{r}_i) + k_{p,i}(r_{i,T} - r_i) + k_{i,i} \int (r_{i,T} - r_i) = 0,$$

Desired acceleration obtained from above is used to obtain desired pitch, and roll angles via linearizing the dynamics:

$$\begin{aligned} \ddot{r}_1^{\text{des}} &= g(\theta^{\text{des}} \cos \psi_T + \phi^{\text{des}} \sin \psi_T), \\ \ddot{r}_2^{\text{des}} &= g(\theta^{\text{des}} \sin \psi_T - \phi^{\text{des}} \cos \psi_T), \\ \ddot{r}_3^{\text{des}} &= \frac{8k_F\omega_h}{m} \Delta\omega_F. \end{aligned} \quad \longrightarrow \quad \begin{aligned} \phi^{\text{des}} &= \frac{1}{g}(\ddot{r}_1^{\text{des}} \sin \psi_T - \ddot{r}_2^{\text{des}} \cos \psi_T), \\ \theta^{\text{des}} &= \frac{1}{g}(\ddot{r}_1^{\text{des}} \cos \psi_T + \ddot{r}_2^{\text{des}} \sin \psi_T), \\ \Delta\omega_F &= \frac{m}{8k_F\omega_h} \ddot{r}_3^{\text{des}}. \end{aligned}$$

Quadcopter Control – Attitude Control

- Once desired attitudes are obtained, they are controlled using PD controller:

$$\Delta\omega_\phi = k_{p,\phi}(\phi^{\text{des}} - \phi) + k_{d,\phi}(\dot{\phi}^{\text{des}} - \dot{\phi}),$$

$$\Delta\omega_\theta = k_{p,\theta}(\theta^{\text{des}} - \theta) + k_{d,\theta}(\dot{\theta}^{\text{des}} - \dot{\theta}),$$

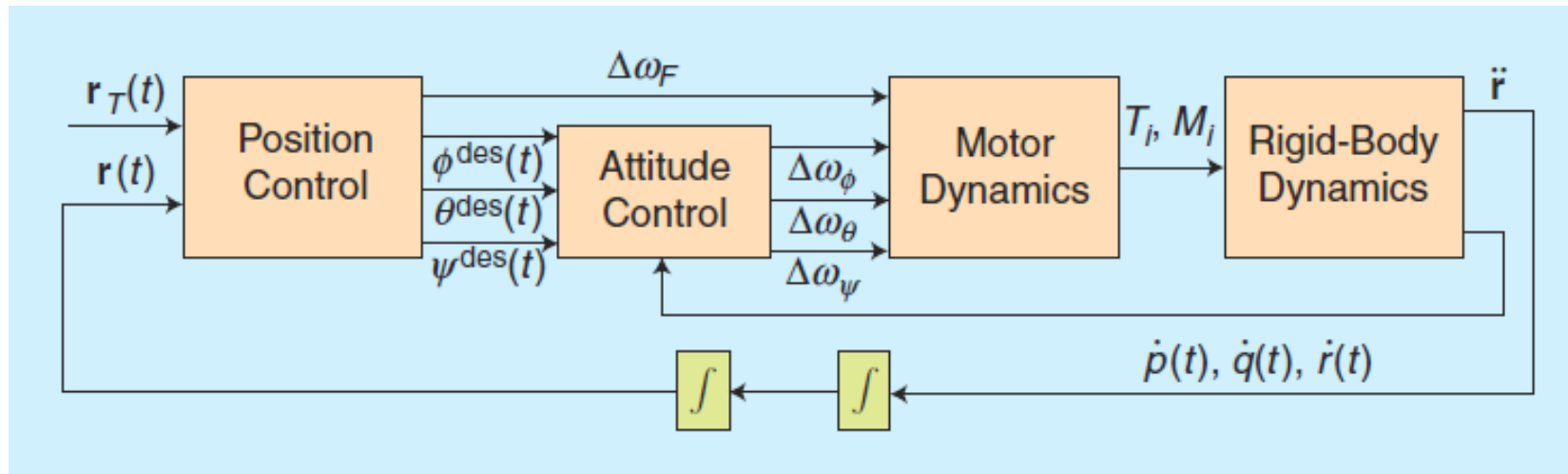
$$\Delta\omega_\psi = k_{p,\psi}(\psi^{\text{des}} - \psi) + k_{d,\psi}(\dot{\psi}^{\text{des}} - \dot{\psi}).$$

As near the nominal hover state, $\dot{\phi} \approx \dot{p}$, $\dot{\theta} \approx \dot{q}$, and $\dot{\psi} \approx \dot{r}$,

Then, desired rotational speeds are obtained using:

$$\begin{bmatrix} \omega_1^{\text{des}} \\ \omega_2^{\text{des}} \\ \omega_3^{\text{des}} \\ \omega_4^{\text{des}} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -1 & 1 \\ 1 & 1 & 0 & -1 \\ 1 & 0 & 1 & 1 \\ 1 & -1 & 0 & -1 \end{bmatrix} \begin{bmatrix} \omega_h + \Delta\omega_F \\ \Delta\omega_\phi \\ \Delta\omega_\theta \\ \Delta\omega_\psi \end{bmatrix},$$

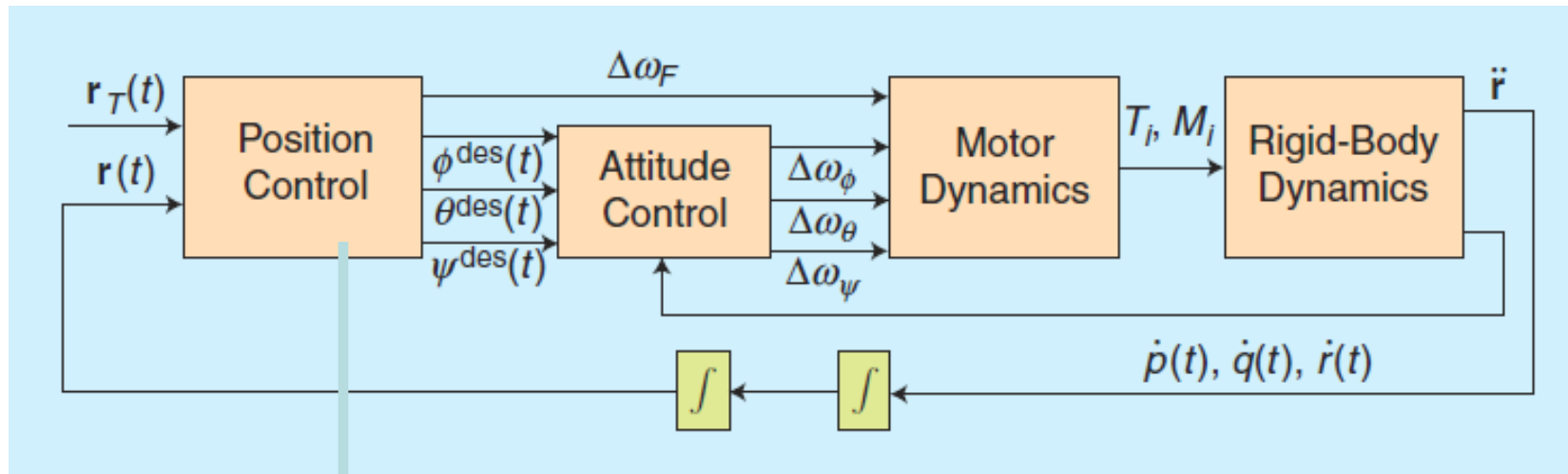
Quadrotor Control



Nested Control Loop for Position and Attitude Control

Quadrotor Control

Nested Control Loop for Position and Attitude Control



$$\phi^{\text{des}} = \frac{1}{g} (\ddot{r}_1^{\text{des}} \sin \psi_T - \ddot{r}_2^{\text{des}} \cos \psi_T),$$

$$\theta^{\text{des}} = \frac{1}{g} (\ddot{r}_1^{\text{des}} \cos \psi_T + \ddot{r}_2^{\text{des}} \sin \psi_T),$$

$$\Delta \omega_F = \frac{m}{8k_F \omega_h} \ddot{r}_3^{\text{des}}.$$

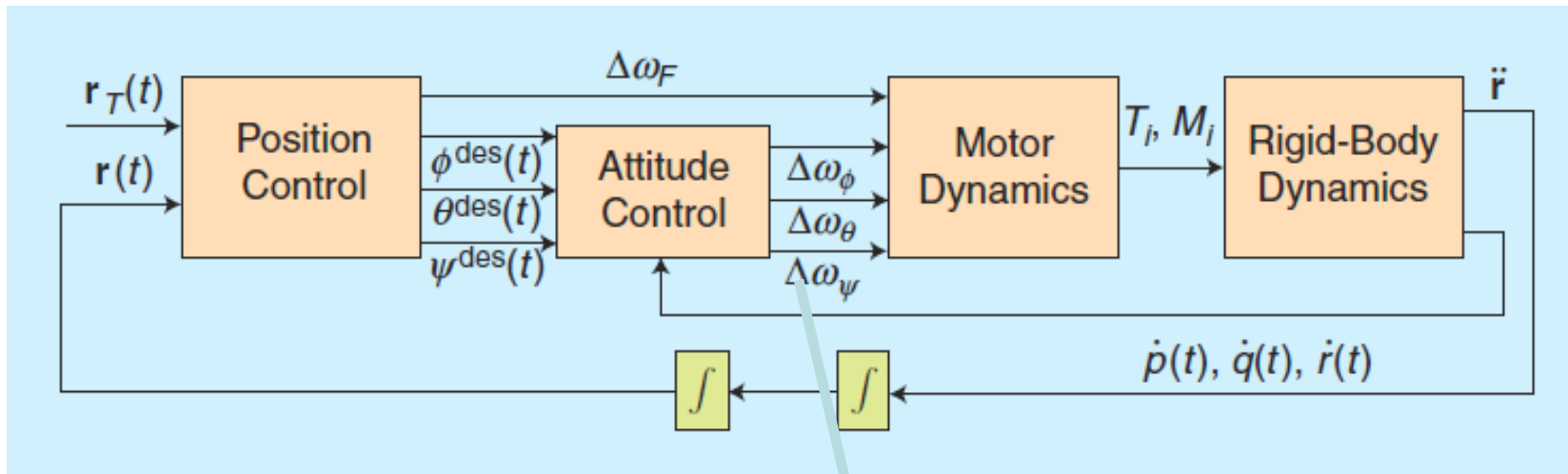
$$(\ddot{r}_{i,T} - \ddot{r}_i^{\text{des}}) + k_{d,i}(\dot{r}_{i,T} - \dot{r}_i) + k_{p,i}(r_{i,T} - r_i) + k_{i,i} \int (r_{i,T} - r_i) = 0,$$

$$\ddot{r}_1^{\text{des}} = g(\theta^{\text{des}} \cos \psi_T + \phi^{\text{des}} \sin \psi_T),$$

$$\ddot{r}_2^{\text{des}} = g(\theta^{\text{des}} \sin \psi_T - \phi^{\text{des}} \cos \psi_T),$$

$$\ddot{r}_3^{\text{des}} = \frac{8k_F \omega_h}{m} \Delta \omega_F.$$

Quadrotor Control



Nested Control Loop for Position and Attitude Control

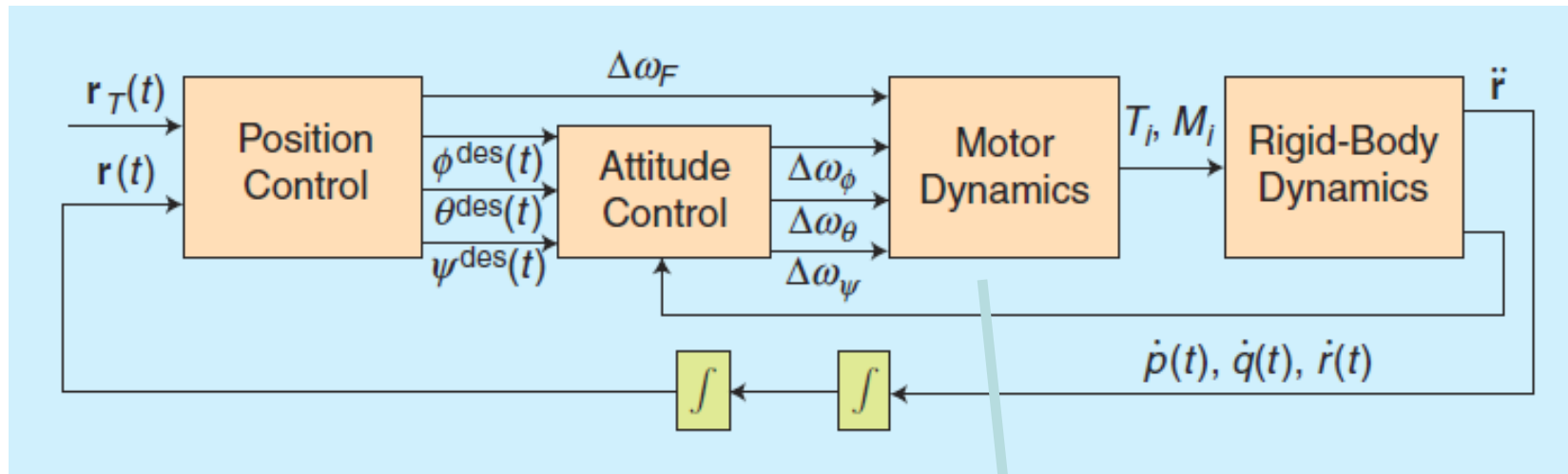
$$\Delta\omega_\phi = k_{p,\phi}(\phi^{des} - \phi) + k_{d,\phi}(\dot{\phi}^{des} - \dot{\phi}),$$

$$\Delta\omega_\theta = k_{p,\theta}(\theta^{des} - \theta) + k_{d,\theta}(\dot{\theta}^{des} - \dot{\theta}),$$

$$\Delta\omega_\psi = k_{p,\psi}(\psi^{des} - \psi) + k_{d,\psi}(\dot{\psi}^{des} - \dot{\psi}).$$

Quadrotor Control

Nested Control Loop for Position and Attitude Control

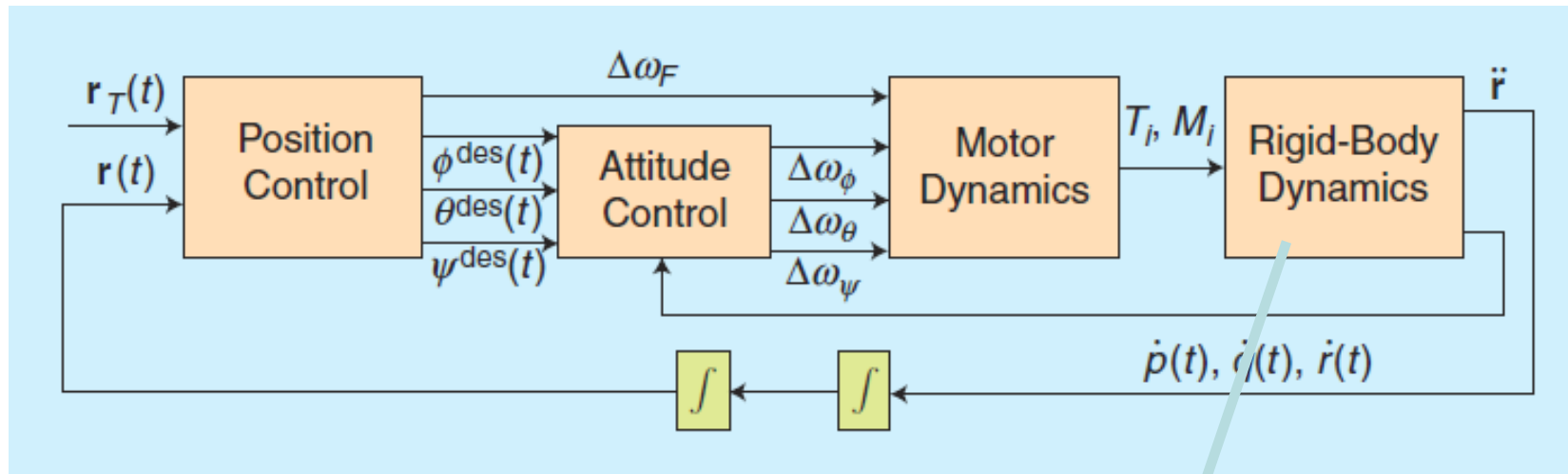


$$\begin{bmatrix} \omega_1^{\text{des}} \\ \omega_2^{\text{des}} \\ \omega_3^{\text{des}} \\ \omega_4^{\text{des}} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -1 & 1 \\ 1 & 1 & 0 & -1 \\ 1 & 0 & 1 & 1 \\ 1 & -1 & 0 & -1 \end{bmatrix} \begin{bmatrix} \omega_h + \Delta\omega_F \\ \Delta\omega_\phi \\ \Delta\omega_\theta \\ \Delta\omega_\psi \end{bmatrix}$$

$$F_i = k_F \omega_i^2 \quad M_i = k_M \omega_i^2$$

Quadrotor Control

Nested Control Loop for Position and Attitude Control



$$m\ddot{\mathbf{r}} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + R \begin{bmatrix} 0 \\ 0 \\ \Sigma F_i \end{bmatrix}$$

$$\begin{aligned} I_{xx}\dot{p} &= Lk_F(\omega_2^2 - \omega_4^2) - qr(I_{zz} - I_{yy}), \\ I_{yy}\dot{q} &= Lk_F(\omega_3^2 - \omega_1^2) - pr(I_{xx} - I_{zz}), \\ I_{zz}\dot{r} &= k_M(\omega_1^2 - \omega_2^2 + \omega_3^2 - \omega_4^2). \end{aligned}$$